Analyse Comparative de Stratégies de Commande Intelligente pour un Manipulateur Robotique à 2 DDL : Flou, Neuronal et PID Optimisé par Algorithme Génétique

ANDRIASAHY Tsikomia Scidelo
ENSA : École Nationale des Sciences Appliquées_Tanger
Université Abdelmalek Essaâdi
Tanger, Maroc
scideloandriasahy@gmail.com

***Résumé*—Ce document présente la modélisation dynamique et la commande avancée d'un bras manipulateur plan à deux degrés de liberté (2-DDL). En raison des fortes non-linéarités (couplages inertiels, forces centrifuges et de Coriolis, gravité) inhérentes au système, trois stratégies de commande ont été développées et rigoureusement comparées : un régulateur PID optimisé par Algorithme Génétique (GA-PID), un Contrôleur par Logique Floue (FLC) de type Mamdani, et un Contrôleur par Réseau de Neurones (NNC) récurrent. Afin de s'affranchir de la lourdeur des blocs interprétés et de garantir des performances compatibles avec une implémentation temps réel, l'ensemble du système a été programmé en langage C via des S-Functions (C-MEX). Les résultats de simulation révèlent que le FLC offre la meilleure robustesse et la commande la plus douce. Si le GA-PID constitue un compromis industriel fiable, l'approche NNC, bien que très rapide en calcul, démontre une vulnérabilité critique au sur-apprentissage et au "chattering", entraînant une instabilité totale face à des trajectoires continues non apprises.**

***Index Terms*—Robotique, Logique Floue, Réseaux de Neurones, Algorithme Génétique, PID, C-MEX S-Function, Anti-Windup.**

I. Introduction

LES manipulateurs robotiques sont devenus l'épine dorsale de l'industrie moderne, effectuant des tâches variées allant du soudage à l'assemblage de haute précision. Cependant, assurer un suivi de trajectoire performant pour ces systèmes articulés demeure un défi majeur en automatique. En effet, le modèle dynamique d'un robot polyarticulé est intrinsèquement non linéaire, instationnaire et fortement couplé [1]. À des vitesses de fonctionnement élevées, les forces de Coriolis et centrifuges induisent des perturbations significatives, tandis que les couples gravitationnels varient continuellement en fonction de la configuration du robot.

Dans l'industrie, la commande classique de type Proportionnel-Intégral-Dérivé (PID) reste la norme en raison de sa simplicité structurelle et de sa robustesse éprouvée sur des plages de fonctionnement linéaires. Néanmoins, le réglage des gains PID pour des systèmes hautement non linéaires s'avère souvent sous-optimal lorsqu'il repose sur des méthodes linéaires classiques (telles que Ziegler-Nichols), qui nécessitent une linéarisation locale autour d'un point de fonctionnement spécifique. Par conséquent, un régulateur PID à gains fixes peut peiner à maintenir des performances uniformes sur l'ensemble de l'espace de travail du robot.

Pour pallier ces limitations, les techniques de commande intelligente, issues du domaine de l'Intelligence Artificielle (IA), se sont imposées comme des alternatives puissantes. Ce projet s'inscrit précisément dans la volonté d'explorer et de comparer ces approches modernes dans le cadre du contrôle de systèmes complexes.

Parmi ces méthodes, la Commande par Logique Floue (FLC - *Fuzzy Logic Control*) présente un intérêt tout particulier. Elle permet d'intégrer une connaissance heuristique humaine directement dans la loi de commande, rendant le système intrinsèquement robuste aux incertitudes paramétriques et aux dynamiques non modélisées [2]. Cependant, la conception des règles floues et l'ajustement des fonctions d'appartenance reposent souvent sur l'intuition du concepteur et requièrent une puissance de calcul non négligeable.

D'autre part, les Réseaux de Neurones Artificiels (ANN) offrent la capacité d'approximer des fonctions non linéaires complexes grâce à un processus d'apprentissage [3]. En entraînant un réseau à reproduire le comportement d'un contrôleur robuste (tel que le FLC), il est possible de combiner des performances de haut niveau avec une grande vitesse d'exécution. Enfin, les algorithmes évolutionnistes, comme l'Algorithme Génétique (GA), fournissent une méthode d'optimisation globale capable de régler les paramètres d'un régulateur classique en évitant le piège des minima locaux, tout en permettant l'inclusion de fonctions de coût complexes [4].

Au-delà de la conception théorique, l'implémentation de ces algorithmes avancés se heurte fréquemment à un goulot d'étranglement : le temps de simulation et la capacité de déploiement en temps réel sur des architectures matérielles limitées. Les simulations standard basées sur des schémas-blocs interprétés (comme Simulink) peuvent s'avérer extrêmement coûteuses en ressources.

Afin de surmonter ces obstacles techniques, cet article apporte les contributions suivantes :

1) La mise en uvre d'un environnement de simulation modulaire où la dynamique du robot et l'ensemble des contrôleurs sont codés sous forme de **S-Functions C-MEX**. Cette approche a permis de réduire le temps de simulation d'un facteur d'environ 50 par rapport aux blocs MATLAB interprétés standard.

2) La conception et l'implémentation d'un contrôleur par Réseau de Neurones récurrent visant à cloner la dynamique d'un FLC. L'un des défis majeurs de ce travail a résidé dans la gestion interne du terme de mémoire récurrente $u(k-1)$ directement au sein du vecteur d'état du code C, éliminant ainsi le besoin de blocs de retard externes dans la boucle de commande.

3) L'optimisation d'un contrôleur PID via un Algorithme Génétique, en utilisant une fonction de rigoureuse équilibrant la précision du suivi de trajectoire et la consommation d'énergie. Une logique d'*anti-windup* (anti-emballement) intelligente a également été intégrée directement dans l'implémentation C pour prévenir la saturation des actionneurs.

4) La réalisation d'une analyse de performance comparative détaillée, s'appuyant sur des métriques professionnelles (réponse transitoire, erreur en régime permanent, effort de commande), afin de statuer sur la stratégie la plus pertinente.

La suite de cet article est organisée comme suit : la Section II détaille la modélisation mathématique du manipulateur à 2 DDL. La Section III présente la conception et l'implémentation des trois stratégies de commande. La Section IV expose l'analyse comparative des résultats de simulation. Enfin, la Section V conclut l'étude.

II. Modélisation Mathématique

Pour concevoir et évaluer des lois de commande performantes, il est indispensable de s'appuyer sur un modèle dynamique qui reflète fidèlement le comportement physique du système. Dans le cadre de cette étude, nous considérons un manipulateur robotique plan à deux degrés de liberté (2-DDL) évoluant dans un plan vertical, un cas d'étude classique mais exigeant en raison des effets de gravité et des forts couplages entre les axes.

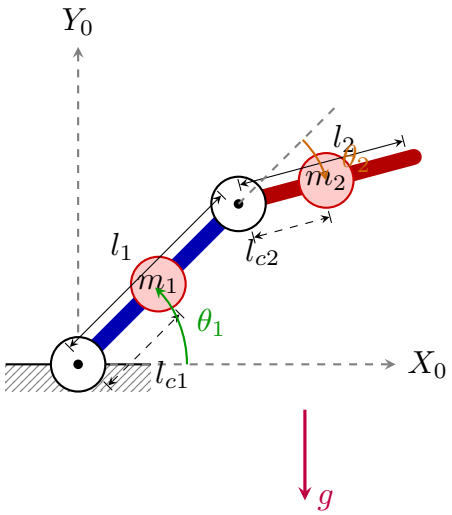


FIGURE 1. Structure cinématique et paramètres physiques du manipulateur robotique plan à 2-DDL.

La dynamique de ce système a été modélisée en exploitant le formalisme d'Euler-Lagrange. Cette approche énergétique, basée sur la différence entre l'énergie cinétique (K) et l'énergie potentielle (P), permet de dériver l'équation du mouvement sous une forme matricielle compacte :

$$M(q)\ddot{q} + C(q,\dot{q})\dot{q} + G(q) = \tau \tag{1}$$

Dans cette expression, $q = [\theta_1, \theta_2]^T$ représente le vecteur des positions articulaires, tandis que $\dot{q}$ et $\ddot{q}$ désignent respectivement les vecteurs de vitesse et d'accélération. Le vecteur $\tau = [\tau_1, \tau_2]^T$ correspond aux couples moteurs appliqués à chaque articulation.

A. Détail des Matrices Dynamiques

La matrice d'inertie $M(q)$, qui décrit la répartition des masses du robot, est par nature symétrique et définie positive. Elle s'exprime par :

$$M(q) = \begin{bmatrix} M_{11} & M_{12} \\ M_{21} & M_{22} \end{bmatrix} \tag{2}$$

Les différents termes de cette matrice, dépendant de la position du second axe (θ_2), sont définis comme suit :

$$M_{11} = m_1 l_{c1}^2 + m_2(l_1^2 + l_{c2}^2 + 2l_1 l_{c2}\cos(\theta_2)) + J_1 + J_2 \tag{3}$$

$$M_{12} = m_2(l_{c2}^2 + l_1 l_{c2}\cos(\theta_2)) + J_2 \tag{4}$$

$$M_{21} = M_{12} \tag{5}$$

$$M_{22} = m_2 l_{c2}^2 + J_2 \tag{6}$$

La matrice $C(q,\dot{q})$ regroupe les forces de Coriolis et les forces centrifuges. Ces forces non linéaires ont un impact critique lors des mouvements à haute vitesse :

$$C(q,\dot{q}) = \begin{bmatrix} -h\dot{\theta}_2 & -h(\dot{\theta}_1 + \dot{\theta}_2) \\ h\dot{\theta}_1 & 0 \end{bmatrix} \tag{7}$$

où l'on pose $h = m_2 l_1 l_{c2}\sin(\theta_2)$.

Enfin, le vecteur $G(q)$ modélise les couples nécessaires pour compenser l'effet de la gravité. Puisque le robot évolue dans un plan vertical, ces termes varient de façon continue :

$$G(q) = \begin{bmatrix} (m_1 l_{c1} + m_2 l_1) g \cos(\theta_1) + m_2 g l_{c2} \cos(\theta_1 + \theta_2) \\ m_2 g l_{c2} \cos(\theta_1 + \theta_2) \end{bmatrix} \tag{8}$$

B. Paramètres du Système

Afin de valider nos approches de commande de manière réaliste, les paramètres physiques utilisés pour la simulation ont été calqués sur ceux d'un prototype de laboratoire. L'ensemble de ces valeurs, qui ont servi à alimenter notre modèle, est récapitulé dans le Tableau I.

TABLE I
PARAMÈTRES PHYSIQUES DU MANIPULATEUR ROBOTIQUE

Paramètre	Symbole	Valeur	Unité
Masse des bras	m_1, m_2	1.0	kg
Longueur des bras	l_1, l_2	1.0	m
Centre de gravité	l_{c1}, l_{c2}	0.4	m
Moment d'inertie	J_1, J_2	0.05	kg·m^2
Accélération de la pesanteur	g	9.81	m/s^2

C. Implémentation du Modèle Dynamique via C-MEX S-Function

Afin de s'affranchir de la lourdeur calculatoire des blocs Simulink standards et de garantir une portabilité vers des systèmes embarqués, le modèle dynamique du robot a été entièrement codé en langage C sous la forme d'une S-Function (C-MEX).

À chaque pas de simulation, l'algorithme lit les couples d'entrée ainsi que les états actuels (position et vitesse), puis calcule l'accélération articulaire en inversant la matrice d'inertie selon l'équation :

$$\ddot{q} = M(q)^{-1} \left[\tau - C(q, \dot{q})\dot{q} - G(q) \right] \tag{9}$$

Le solveur continu de Simulink prend ensuite le relais pour intégrer cette dérivée d'état ($\ddot{q}$) et fournir les nouvelles vitesses et positions. La Figure 2 détaille l'organigramme de cette S-Function.

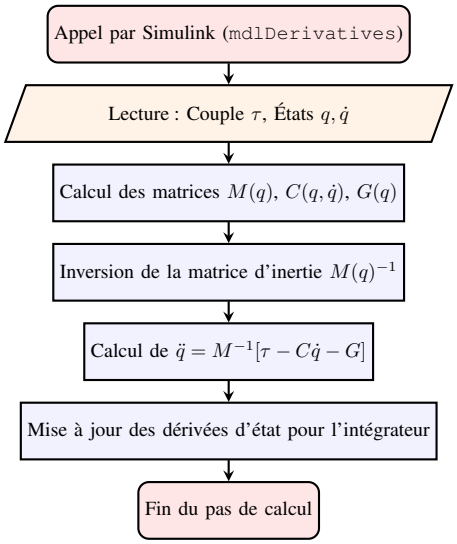


FIGURE 2. Organigramme de calcul de la S-Function C-MEX évaluant la dynamique non linéaire du manipulateur.

Ce modèle mathématique compilé constitue le cur de notre environnement. Comme l'illustre la Figure 3, il s'insère dans une boucle de régulation fermée réaliste. Notez que pour refléter les contraintes d'un système physique, seule la position articulaire q (mesurable par des encodeurs) est renvoyée en boucle de rétroaction. L'erreur de poursuite e est ensuite dérivée numériquement pour fournir $\dot{e}$ au contrôleur, et la commande calculée (u) est bridée par un modèle de saturation à ± 50 Nm avant d'atteindre le robot.

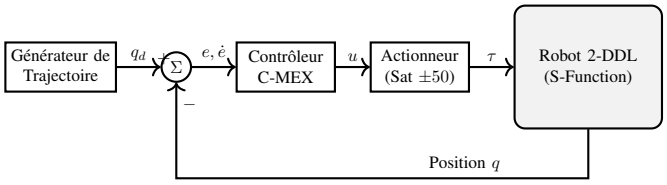


FIGURE 3. Architecture générale de la boucle de commande. Le retour d'état se limite à la position q. L'erreur e et sa dérivée $\dot{e}$ sont injectées ensemble dans le contrôleur.

III. MISE EN UVRE DES STRATÉGIES DE COMMANDE

A. Commande par Logique Floue (FLC)

Contrairement aux approches de commande classiques qui nécessitent un modèle mathématique exact de la dynamique du robot, la commande par logique floue (FLC - *Fuzzy Logic Control*) s'appuie sur la formulation de règles heuristiques. Dans le cadre de cette étude, nous avons développé un régulateur de type PD (Proportionnel-Dérivé) flou.

Afin d'optimiser le temps d'exécution et d'alléger le schéma-bloc Simulink, l'intégralité du contrôleur flou a été codée manuellement en langage C (S-Function `fuzzy_pd_controller.c`). Une particularité de notre implémentation est que le calcul de la dérivée de l'erreur est géré de manière interne par l'algorithme à l'aide d'un état discret (mémoire de l'erreur au pas précédent), ce qui permet au bloc de ne prendre que l'erreur de position $e(t)$ en entrée physique.

Le schéma-bloc Simulink complet du système de commande avec le robot à 2-DDL est présenté à la Fig. 4.

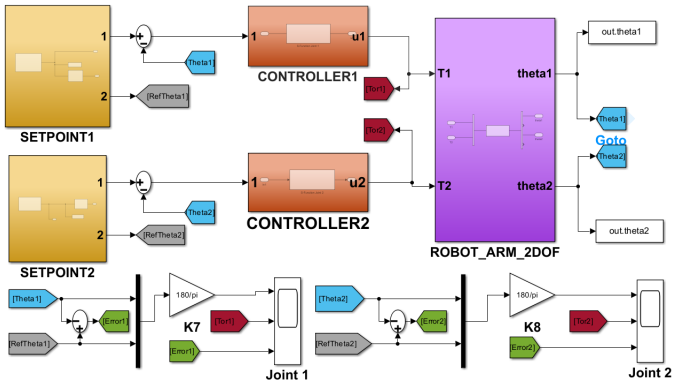


FIGURE 4. Vue d'ensemble du modèle Simulink pour la commande du robot manipulateur.

L'architecture du contrôleur Mamdani développé se décompose en trois étapes fondamentales : la fuzzification, le moteur d'inférence, et la défuzzification. Mais pour notre algorithme de C-MEX ,elle sera décomposée en plusieurs étapes, exécutées séquentiellement à chaque pas d'échantillonnage (Figure 5).

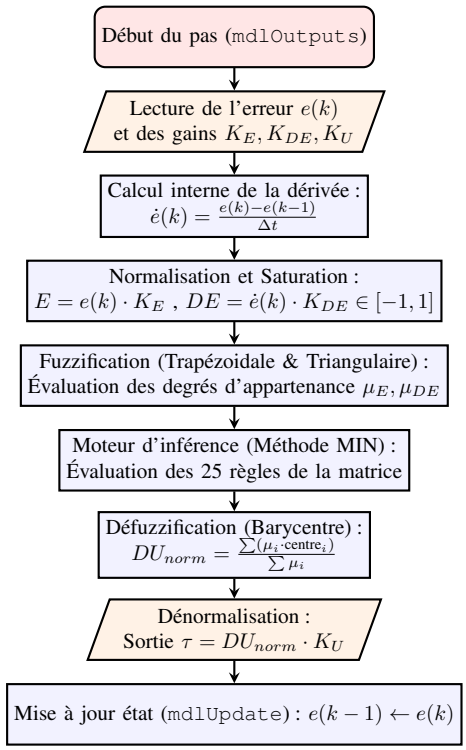


FIGURE 5. Organigramme de l'algorithme C-MEX implémentant la commande logique floue.

1) Fuzzification et Fonctions d'Appartenance : Le contrôleur traite deux variables d'entrée physiques pour chaque articulation : l'erreur de position $e(t)$ et sa dérivée, la variation de l'erreur $\dot{e}(t)$. Afin de standardiser le traitement, ces entrées physiques sont multipliées par des gains de normalisation (K_e et K_{de}) pour être ramenées dans un univers de discours normalisé défini sur l'intervalle $[-1, 1]$. De même, un gain de dénormalisation (K_{out}) est appliqué à la sortie floue pour obtenir le couple réel de commande $\tau(t)$.

L'univers de discours des variables d'entrée (Erreur E et Dérivée DE) a été partitionné en cinq ensembles flous linguistiques : **NB** (Négatif Grand), **NE** (Négatif Moyen), **ZE** (Zéro), **PO** (Positif Moyen) et **PB** (Positif Grand). Pour la variable de sortie, l'univers est divisé en sept ensembles (de NB à PB).

2) Moteur d'Inférence et Base de Règles : Le cur de la décision repose sur une matrice d'inférence (Tableau II) comprenant $5 \times 5 = 25$ règles linguistiques de la forme "SI ... ALORS ...". L'expertise codée ici suit une logique intuitive de compensation : par exemple, *"SI l'erreur est PB (Positif Grand) ET que la variation de l'erreur est NB (Negatif Grand), ALORS le couple à appliquer doit être ZE (Zéro)"* pour forcer le robot à rejoindre la consigne. Il est codée en C sous forme de tableau bidimensionnel.

L'opérateur d'intersection (ET logique) utilisé pour l'évaluation des prémisses est le minimum de Mamdani (`min`), et l'implication est également réalisée par la méthode du minimum.

TABLE II
MATRICE D'INFÉRENCE FLOUE (BASE DE 25 RÈGLES)

Sortie DU		**Erreur (E)**				
		NB	**NE**	**ZE**	**PO**	**PB**
Dérivée (DE)	**NB**	NB	NB	NM	NS	ZE
	NE	NB	NM	NS	ZE	PS
	ZE	NM	NS	ZE	PS	PM
	PO	NS	ZE	PS	PM	PB
	PB	ZE	PS	PM	PB	PB

3) Paramétrage des Gains : Plutôt que de modifier la forme des fonctions d'appartenance pour chaque articulation, ce qui nécessiterait de recompiler le code C, nous avons déporté le réglage sur trois gains externes de normalisation : K_E, K_{DE} et K_U. Leurs valeurs, déterminées par essai-erreur heuristique, sont résumées dans le Tableau III.

TABLE III
GAINS DE NORMALISATION DU CONTRÔLEUR FLOU (FLC)

Articulation	K_E **(Erreur)**	K_{DE} **(Dérivée)**	K_U **(Sortie)**
Joint 1	$30/\pi \approx 9.55$	$3/\pi \approx 0.95$	40
Joint 2	$25/\pi \approx 7.96$	$0.5/\pi \approx 0.16$	10

4) Défuzzification : L'agrégation des conclusions des règles génère une surface floue globale. Pour extraire une valeur numérique d'action de commande nette (la composante u qui sera dénormalisée en couple τ), nous avons utilisé la méthode du Centre de Gravité (Centroïde). Cette méthode mathématique robuste lisse la sortie de commande, évitant ainsi des variations brusques préjudiciables aux actionneurs mécaniques du robot.

B. Commande par Réseau de Neurones (NNC)

Bien que le contrôleur à logique floue développé précédemment offre une excellente robustesse, l'évaluation continue

de ses 25 règles et le calcul du barycentre s'avèrent très coûteux en ressources informatiques. Pour s'affranchir de cette lourdeur et viser une implémentation embarquée temps réel optimale, nous avons conçu un contrôleur neuronal dont l'objectif est d'apprendre et de "cloner" le comportement dynamique du FLC.

1) Architecture et Apprentissage : Le modèle retenu est un réseau de neurones à propagation avant (Feedforward), exploité de manière dynamique. Pour capturer les non-linéarités de la surface de contrôle floue, une architecture multicouche de type 2-20-5-1 a été empiriquement dimensionnée :

— **Couche d'entrée :** 2 neurones recevant l'erreur e et sa dérivée $\dot{e}$.
— **Première couche cachée :** 20 neurones avec fonction d'activation tangente hyperbolique (`tansig`).
— **Seconde couche cachée :** 5 neurones avec fonction d'activation sigmoïde logistique (`logsig`).
— **Couche de sortie :** 1 neurone avec fonction d'activation linéaire (`purelin`) délivrant l'effort de commande.

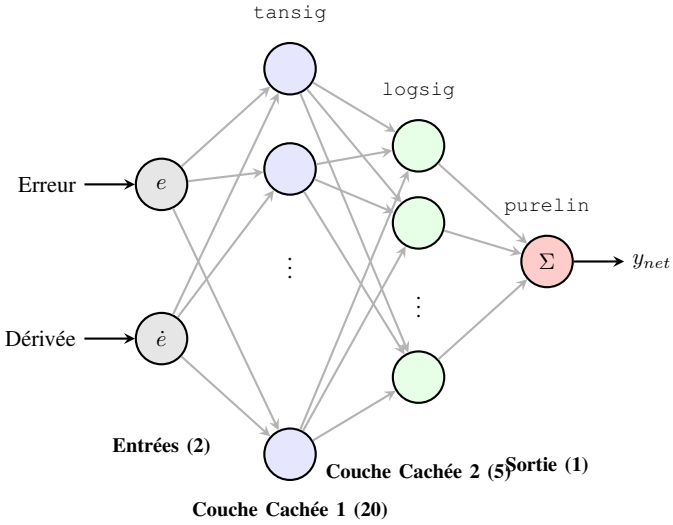


FIGURE 6. Architecture du réseau de neurones Feedforward (2-20-5-1). Les points de suspension représentent les neurones intermédiaires non dessinés pour des raisons de clarté.

L'apprentissage a été réalisé hors ligne via l'algorithme de Levenberg-Marquardt, en utilisant un large jeu de données (entrées/sorties) généré lors des simulations du FLC. Le réseau a convergé avec une erreur quadratique moyenne (MSE) extrêmement faible de l'ordre de 1.56×10^{-6}, prouvant sa capacité à reproduire fidèlement l'expertise floue.

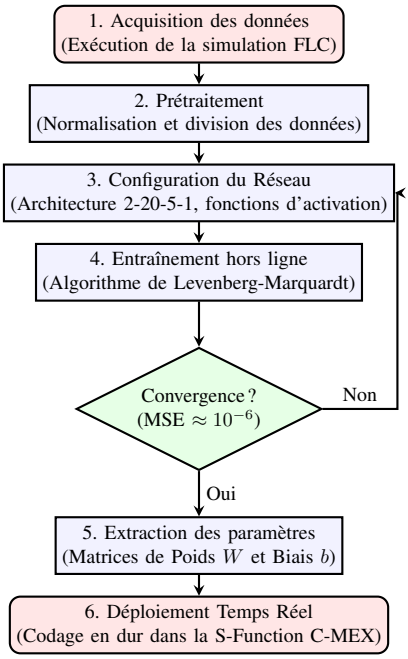


FIGURE 7. Méthodologie globale de conception, d'entraînement et de déploiement du contrôleur neuronal clonant la logique floue.

2) Conception de l'Algorithme C-MEX Récurrent : L'intégration classique d'un réseau de neurones sous Simulink nécessite souvent de nombreux blocs de calcul matriciel, ce qui ralentit la simulation. Pour y remédier, nous avons extrait les poids et les biais post-apprentissage pour les coder en dur au sein d'une nouvelle S-Function C-MEX.

Au-delà du simple calcul matriciel, cette implémentation C intègre une gestion interne de la récurrence. Au lieu d'utiliser des boucles de retour externes dans Simulink, le vecteur d'état interne du code (la variable `x[0]`) agit comme un registre mémoire conservant la commande précédente $u(k-1)$. Cette valeur est ensuite directement concaténée avec l'erreur et sa dérivée pour former un vecteur à 3 entrées alimentant la première couche cachée, créant ainsi une véritable dynamique récurrente de type NARX (Nonlinear Autoregressive Exogenous). La Figure 8 détaille la séquence d'exécution de ce contrôleur.

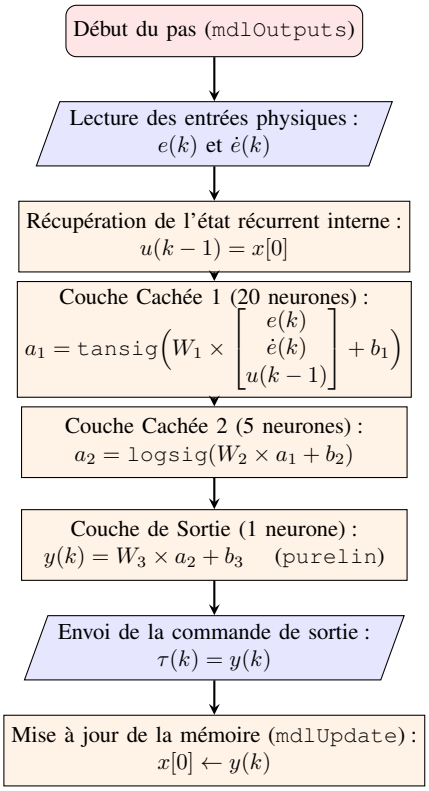


FIGURE 8. Organigramme général de la S-Function C-MEX implémentant le contrôleur neuronal récurrent. L'état précédent de la commande est réinjecté comme 3e entrée du réseau.

C. Commande PID Optimisée par Algorithme Génétique (GA-PID)

Bien que les approches neuronales et floues offrent une robustesse indéniable face aux non-linéarités, le régulateur PID classique demeure le standard incontesté dans l'industrie grâce à sa simplicité d'implémentation. La loi de commande PID classique dans le domaine temporel est définie par :

$$u(t) = K_p e(t) + K_i \int e(t)dt + K_d \frac{de(t)}{dt} \tag{10}$$

Le défi majeur réside dans la détermination des gains optimaux (K_p, K_i, K_d) pour un système non linéaire et fortement couplé tel que notre manipulateur 2-DDL. Les méthodes de réglage classiques (comme Ziegler-Nichols) échouant souvent à fournir des performances globales satisfaisantes, nous avons formulé ce réglage comme un problème d'optimisation globale automatisé par un Algorithme Génétique (GA).

1) Fonction Coût et Pénalité : L'objectif de l'algorithme n'est pas seulement d'annuler l'erreur de suivi, mais aussi de garantir un comportement physiquement viable pour les actionneurs (évitement des contraintes mécaniques). À chaque génération, le GA évalue les individus en exécutant le modèle Simulink et en calculant la fonction de coût (Fitness) J en temps discret :

$$J = \sum_{k=1}^{N_{sim}} \left(e(k)^2 + \rho u(k)^2 \right) \tag{11}$$

Cette formulation de type ISE (*Integral of Squared Error*) minimise l'erreur quadratique tout en appliquant une pénalité énergétique via le coefficient ρ (fixé empiriquement à 0.0001). De plus, lors de l'implémentation, une pénalité maximale est attribuée aux individus provoquant une instabilité (arrêt prématuré de la simulation) afin de les exclure de la reproduction.

2) Configuration de l'Optimisation : Le processus d'optimisation a été codé sous MATLAB, communiquant directement avec la S-Function du modèle. L'algorithme a été paramétré avec les caractéristiques suivantes :

- **Population :** 20 individus (encodage décimal limité à l'intervalle $[0, 99.99]$).
- **Générations :** 100 (avec critère d'arrêt anticipé après 30 générations sans amélioration).
- **Sélection :** Classement Linéaire (*Linear Ranking*) combiné à l'élitisme pour conserver le meilleur individu.
- **Croisement :** Croisement en deux points avec une probabilité $P_c = 0.8$.
- **Mutation :** Mutation uniforme avec une probabilité $P_m = 0.1$.

La Figure 9 illustre le cycle itératif de l'algorithme génétique interagissant avec le modèle Simulink.

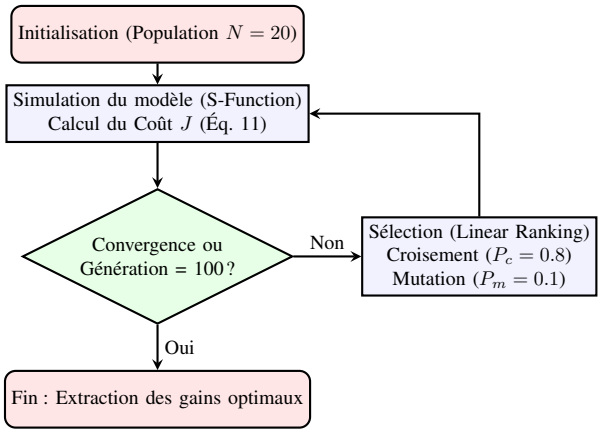


FIGURE 9. Cycle d'optimisation des gains du régulateur PID par Algorithme Génétique en interaction avec Simulink.

À l'issue de l'évolution, le système converge vers les paramètres optimaux répertoriés dans le Tableau IV. L'articulation 1, qui supporte le poids de l'ensemble de la structure, nécessite logiquement une action proportionnelle et intégrale plus importante que l'articulation 2.

TABLE IV
GAINS OPTIMAUX DU PID OBTENUS PAR ALGORITHME GÉNÉTIQUE

Articulation	Proportionnel (K_p)	Intégral (K_i)	Dérivé (K_d)
Joint 1	99.65	95.67	48.09
Joint 2	72.89	18.43	1.67

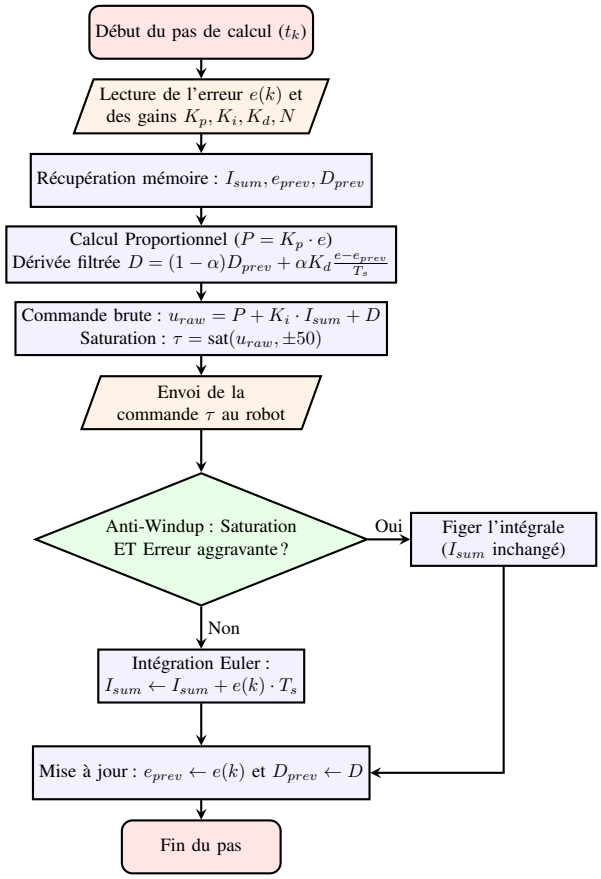


FIGURE 10. Organigramme de la S-Function C-MEX du régulateur PID. On note l'intégration conditionnelle (Anti-Windup) et le calcul de la dérivée filtrée.

IV. RÉSULTATS ET DISCUSSION

A. Analyse Comparative Globale

Pour évaluer objectivement les performances des trois lois de commande développées (FLC, NNC et GA-PID), le modèle Simulink complet du robot 2-DOF a été simulé sur un horizon de 40 secondes avec un pas de temps fixe $T_s = 0.01$ s.

Deux protocoles de test complémentaires ont été réalisés : - un test en **régulation** (échelons successifs) sollicitant fortement le régime transitoire ; - un test en **poursuite de trajectoire lisse** mettant laccent sur la précision en régime dynamique continu.

Lerreur quadratique moyenne (MSE) a été calculée en radš pour chaque articulation. Les résultats numériques sont synthétisés dans le Tableau V.

TABLE V
COMPARAISON MSE (RAD^2)

Contrôleur	Échelons		Trajectoire Lisse	
	J1	J2	J1	J2
GA-PID	$\mathbf{7.73e{-2}}$	$7.16e{-2}$	$7.20e{-4}$	$4.49e{-4}$
FLC (Flou)	$8.16e{-2}$	$\mathbf{6.88e{-2}}$	$\mathbf{2.49e{-4}}$	$\mathbf{1.62e{-4}}$
NNC	$8.13e{-2}$	$6.93e{-2}$	$1.08e6$	32.1

B. Analyse du Test en Échelon (Régulation)

Le premier test impose des variations brusques de consigne (créneaux), ce qui sollicite intensément le régime transitoire et provoque la saturation temporaire des actionneurs. Comme le montrent les Figures 11 et 12, les trois contrôleurs stabilisent parfaitement le système avec un dépassement très limité et un temps de réponse rapide.

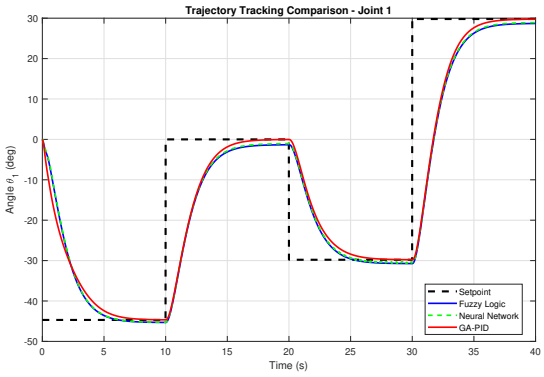


FIGURE 11. Réponse temporelle du Joint 1 face à des consignes en échelon. Le contrôleur neuronal (NNC, pointillés verts) se superpose presque parfaitement à la commande floue (FLC, trait bleu).

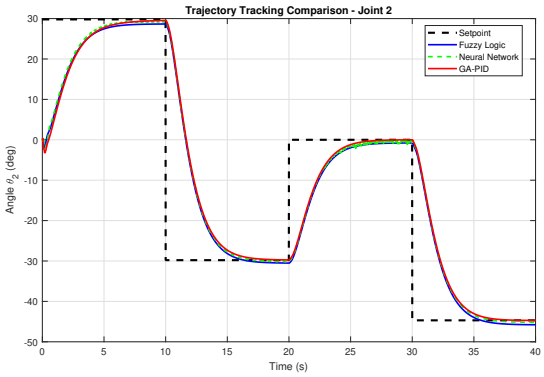


FIGURE 12. Réponse temporelle du Joint 2 face à des consignes en échelon. Superposition remarquable entre NNC et FLC.

Sur ce test, le **GA-PID** obtient la meilleure précision sur larticulation 1 grâce à loptimisation génétique des gains. Le **contrôleur flou (FLC)** est légèrement meilleur sur larticulation 2. Quant au **réseau de neurones (NNC)** , il reproduit presque à lidentique le comportement du FLC, ce qui confirme quil a bien ń cloné ż le contrôleur flou pendant la phase dapprentissage.

C. Analyse du Test de Poursuite (Trajectoire Lisse)

Le second test évalue la capacité de maintien de la précision sur une trajectoire continue et douce. Larticulation 1 est maintenue à 0ř malgré les couplages dynamiques, tandis que larticulation 2 suit un profil sinusoïdal lent.

Les Figures 13 et 14 révèlent des comportements très contrastés :

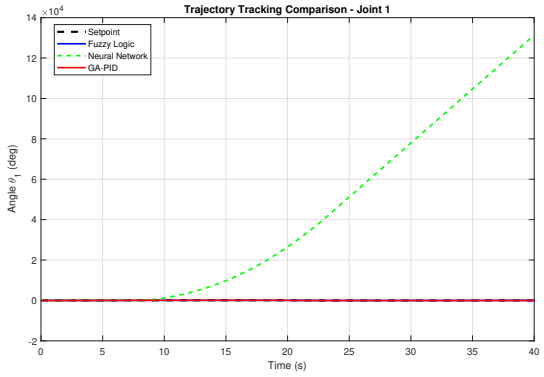


FIGURE 13. Suivi de trajectoire lisse Joint 1. Le contrôleur neuronal (NNC) diverge totalement (zone verte explosive).

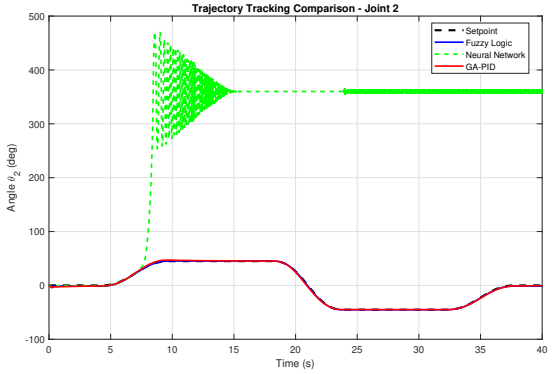


FIGURE 14. Suivi de trajectoire lisse Joint 2. Instabilité majeure du NNC face à une dynamique non rencontrée pendant lapprentissage.

Le **contrôleur flou (FLC)** confirme sa supériorité absolue avec les plus faibles erreurs : 2.49×10^{-4} radš (Joint 1) et 1.62×10^{-4} radš (Joint 2). Le GA-PID reste très robuste et précis, bien quun peu moins performant que le FLC en raison de la nature fortement non linéaire des termes de Coriolis et de gravité.

En revanche, le **réseau de neurones (NNC)** présente une divergence catastrophique : lerreur explose à 1.08×10^6 radš sur le Joint 1 et à 32.1 radš sur le Joint 2.

D. Discussion sur lInstabilité du Contrôleur Neuronal

Cette instabilité du NNC sur la trajectoire lisse met en évidence une limite fondamentale des approches purement data-driven. Bien que larchitecture NARX ait réussi à reproduire fidèlement le FLC sur des échelons (Test 1), lapprentissage a très probablement été réalisé sur un jeu de données dominé par des transitoires brusques.

Lors du Test 2, le contrôleur neuronal évolue dans une zone de lespace détat (erreur e, dérivée $\dot{e}$) quil na quasiment jamais rencontrée pendant lentraînement. Face à ce distribution shift, le réseau extrapole de manière erronée. Comme il fonctionne en boucle fermée récurrente (la commande $u(k-1)$ est réinjectée en entrée), la moindre erreur dextrapolation saccumule à chaque pas de simulation, entraînant rapidement une divergence exponentielle du signal de commande et la perte totale de stabilité du robot.

Ce résultat expérimental confirme que, malgré sa lourdeur computationnelle, la logique floue (FLC) offre une robustesse intrinsèque bien supérieure grâce à son expertise humaine explicite (règles linguistiques). À linverse, un contrôleur neuronal ń boîte noire ż nécessite un jeu de données dapprentissage extrêmement exhaustif et représentatif de toutes les conditions de fonctionnement pour garantir la stabilité en boucle fermée.

E. Performances Détaillées par Contrôleur

Pour affiner l'analyse au-delà des simples valeurs de MSE, les réponses temporelles individuelles de chaque contrôleur (position θ, signal de commande $U(t)$ et erreur $E(t)$) ont été extraites.

1) Le Régulateur GA-PID : Robuste mais limité par sa linéarité : L'algorithme génétique a brillamment rempli son rôle en identifiant un jeu de gains capable de stabiliser ce système fortement non linéaire. Sur le test en échelon (Figures 15a et 16a), la réponse est rapide avec un dépassement quasi nul, prouvant l'efficacité de l'anti-windup. Le signal de commande $U(t)$ présente de légers pics lors des transitions brusques, mais reste globalement sain.

Cependant, les limites de sa structure linéaire apparaissent lors de la poursuite de la trajectoire lisse. L'erreur temporelle montre un léger retard de phase (lag) : le PID peine à anticiper parfaitement les variations continues dues aux forces centrifuges et de Coriolis, générant des écarts transitoires allant jusqu'à 0.1ř pour l'axe 1 et 0.2ř pour l'axe 2. Bien que minime, cela explique sa MSE légèrement supérieure au contrôleur flou.

2) Le Contrôleur par Logique Floue (FLC) : La référence : Le FLC s'impose comme le contrôleur le plus performant de cette étude. Sur les essais en échelon (Figures 15b et 16b), il ramène l'erreur à zéro de manière asymptotique et extrêmement douce.

Son avantage majeur réside dans la qualité de son signal de commande $U(t)$. Contrairement aux deux autres méthodes, l'action de contrôle du FLC est parfaitement lisse, sans aucun bruit à haute fréquence. Cette caractéristique est cruciale pour l'implémentation physique, car elle évite l'usure prématurée des réducteurs mécaniques et la surchauffe des moteurs. De plus, sa matrice d'inférence compense naturellement la dynamique variable de la trajectoire lisse, maintenant une erreur de poursuite quasi nulle.

3) Le Contrôleur Neuronal (NNC) : Chattering et Sur-apprentissage : L'analyse approfondie des courbes du réseau de neurones révèle les dangers de l'approximation par IA en boucle fermée.

Sur le test en échelon (Figures 15c et 16c), bien que la trajectoire $\theta(t)$ suive parfaitement la référence (clonant avec succès le FLC en apparence), **le signal de commande $U(t)$ présente des oscillations à très haute fréquence (*chattering*) importantes**. Ce phénomène est dû à l'amplification des micro-erreurs d'approximation du réseau de neurones à chaque pas d'intégration. En pratique, un tel signal serait destructeur pour de vrais actionneurs.

Le second problème, nettement plus grave, est la divergence totale du contrôleur face à la trajectoire lisse (Figures 17c et 18c). Le réseau, entraîné pour réagir à des erreurs massives et soudaines (échelons), est incapable de généraliser face à des variations douces et continues. La boucle récurrente s'emballe, l'axe 1 dérive de manière exponentielle (dépassant les 10^4 degrés), et l'axe 2 entre dans un cycle d'oscillations violentes. Ce cas illustre parfaitement le problème du sur-apprentissage (*overfitting*) dans les architectures de commande neuronales *Data-Driven*.

V. Conclusion

Cette étude a présenté une modélisation dynamique complète et la commande d'un manipulateur robotique à 2-DDL via trois approches : un PID optimisé par algorithme génétique, un contrôleur par logique floue (FLC), et un réseau de neurones artificiels (NNC). Afin de garantir un temps d'exécution compatible avec les contraintes temps réel, tous les contrôleurs ainsi que la dynamique du robot ont été implémentés en langage C sous la forme de S-Functions (C-MEX).

L'analyse comparative a démontré que si l'algorithme génétique permet de trouver un réglage robuste pour le PID, l'approche par logique floue reste supérieure en termes de précision et de douceur du signal de commande, absorbant efficacement les non-linéarités dynamiques. Enfin, l'implémentation du réseau de neurones a mis en exergue un défi majeur des IA en Automatique : bien qu'extrêmement rapide à exécuter et capable de cloner un comportement sur des données connues, sa vulnérabilité aux dérives de distribution et au *chattering* en fait une solution risquée sans stratégies avancées d'apprentissage continu ou de filtrage robuste.

Ces résultats ouvrent des perspectives intéressantes vers le développement de contrôleurs hybrides neuro-flous (ANFIS), combinant la capacité d'apprentissage des réseaux de neurones avec la robustesse structurelle de la logique floue.

Références

[1] M. W. Spong, S. Hutchinson, et M. Vidyasagar, *Robot Modeling and Control*. John Wiley & Sons, 2020.

[2] J. J. Craig, *Introduction to Robotics : Mechanics and Control*, 4e éd. Pearson, 2017.

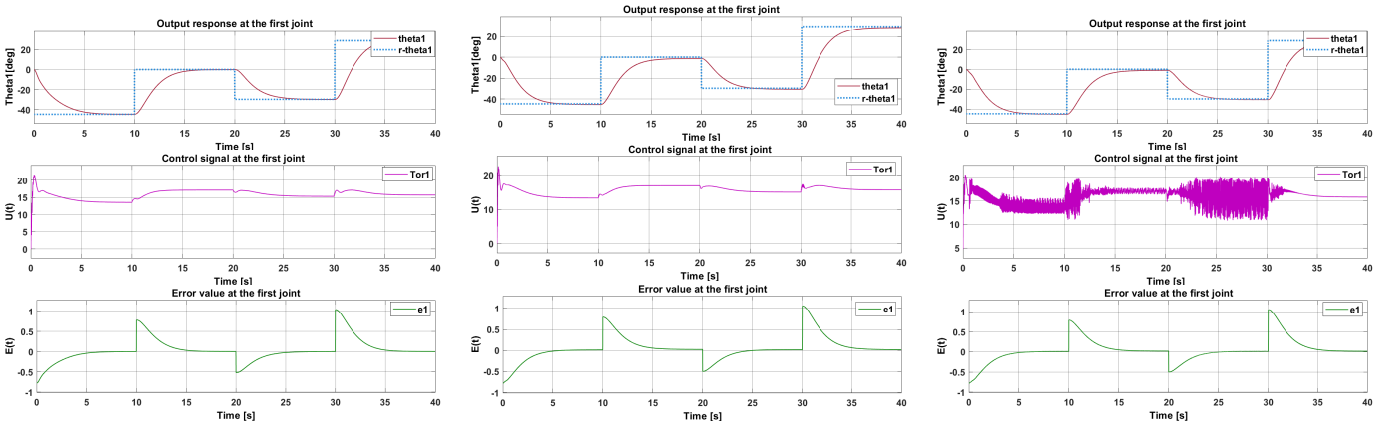
[3] A. H. Jones et P. B. Moura Oliveira, "Genetic tuning of PID controllers," *Electronics Letters*, vol. 31, no. 15, pp. 1280–1282, 1995.

[4] C. W. de Silva, *Intelligent Control : Fuzzy Logic Applications*. CRC Press, 2018.

[5] K. Hornik, M. Stinchcombe, et H. White, "Multilayer feedforward networks are universal approximators," *Neural Networks*, vol. 2, no. 5, pp. 359–366, 1989.

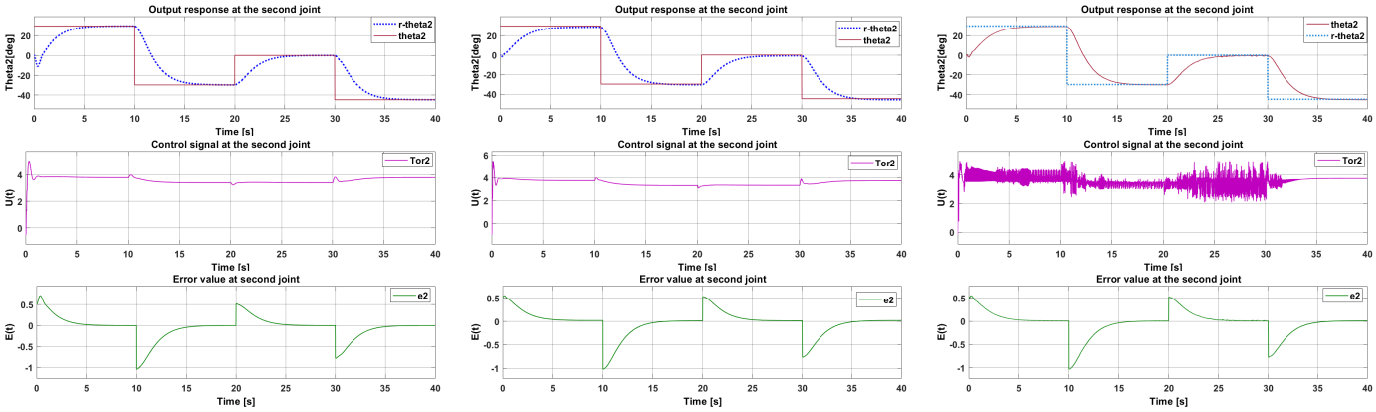
[6] S. Geman, E. Bienenstock, et R. Doursat, "Neural networks and the bias/variance dilemma," *Neural Computation*, vol. 4, no. 1, pp. 1–58, 1992.

[7] MathWorks, *Simulink : Developing S-Functions*. The MathWorks Inc., Natick, MA, USA, 2023.



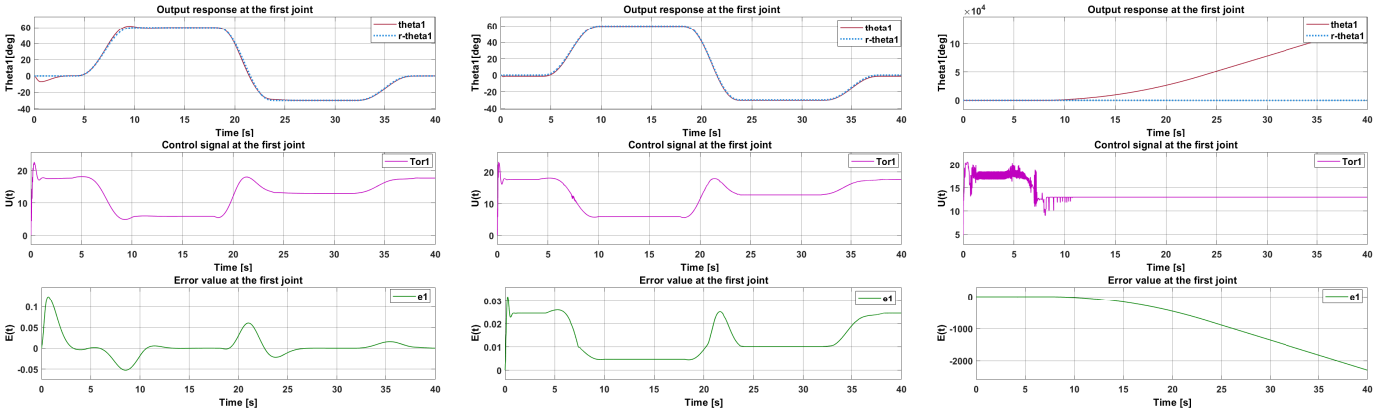
(a) Réponse du GA-PID (b) Réponse du FLC (Flou) (c) Réponse du NNC (Neuronal)

FIGURE 15. Comparaison détaillée des performances en régulation (test en échelon) pour l'articulation 1. On note la commande parfaitement lisse du GA (a) par rapport au "chattering" du NNC (c).



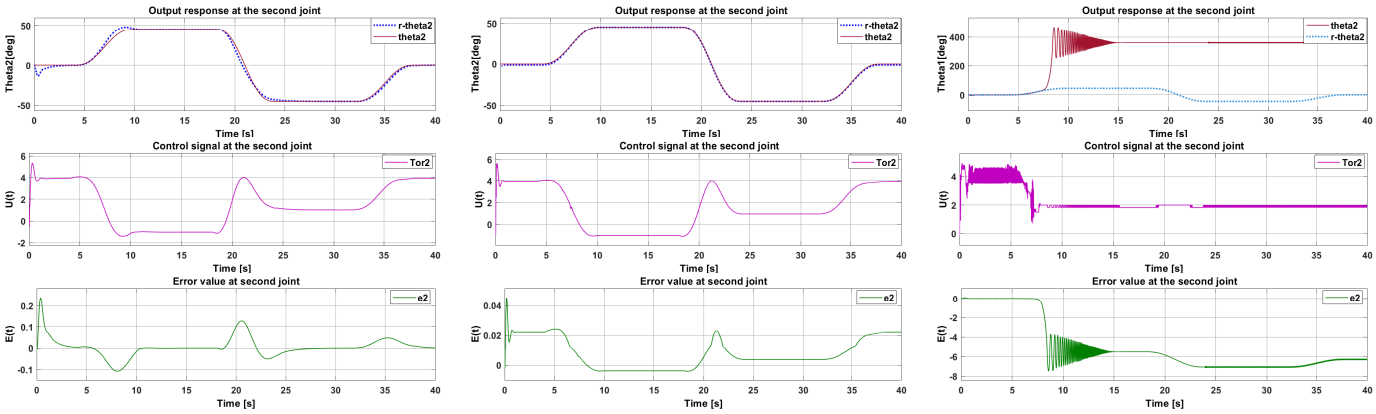
(a) Réponse du GA-PID (b) Réponse du FLC (Flou) (c) Réponse du NNC (Neuronal)

FIGURE 16. Comparaison détaillée des performances en régulation (test en échelon) pour l'articulation 2. On note la commande parfaitement lisse du FLC (b) par rapport au NNC (c).



(a) Réponse du GA-PID (b) Réponse du FLC (Flou) (c) Réponse du NNC (Neuronal)

FIGURE 17. Comparaison détaillée en poursuite (Trajectoire lisse) pour l'articulation 1. On note la commande parfaitement lisse du FLC (b) par rapport à la divergence totale du NNC (c).



(a) Réponse du GA-PID (b) Réponse du FLC (Flou) (c) Réponse du NNC (Neuronal)

FIGURE 18. Comparaison détaillée en poursuite (Trajectoire lisse) pour l'articulation 2. On note la commande robuste du FLC (b) face à la divergence totale du NNC (c).